

Revisão Acelerada de Programação - RAP RAP-2026, Marinha do Brasil

Centro de Instrução Almirante Alexandrino

Departamento de Aperfeiçoamento Avançado para Oficiais

Curso de Aperfeiçoamento Avançado em Guerra Acústica (C-ApA-Of-GA)

Professor: João Victor da Fonseca Pinto

Descrição do Documento: Lista de Exercícios (Parte 2)

Código da Aula: 2

Leia as instruções antes de fazer a lista:

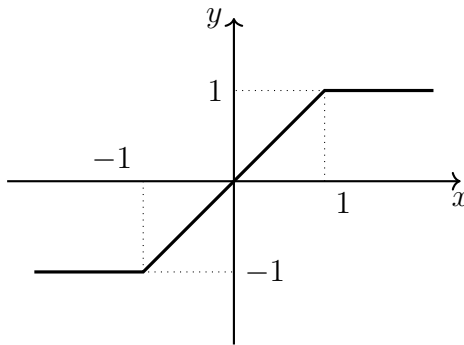
Não utilize laços de repetição ou recursão nessa lista. Não utilize nenhum módulo, método ou função já existente no Python exceto pela função `print`. Não é necessário testar se os dados passados por argumento são válidos a menos que pedido na questão.

- Q1.** Escreva uma função em Python que recebe um número por argumento e retorna o booleano `True` se esse número for do tipo inteiro ou do tipo float, e se o número for positivo. Caso o número não seja nem inteiro nem float, ou se o número for menor ou igual a zero, a função deve retornar o booleano `False`. Utilize operadores lógicos booleanos (`not`, `and` ou `or`).

Resposta:

```
def verificar_numero( num ):  
    if not (type(num)==int or type(num) == float):  
        print("Número não é inteiro ou float.")  
        return False  
  
    if num < 0:  
        print("Número não é positivo")  
        return False  
  
    # número é int,float e positivo (>=0)  
    return True
```

- Q2.** Em ciência da computação, redes neurais são modelos computacionais usados na área de inteligência artificial. A saída dos neurônios é comumente descrita por uma função de ativação, que, muitas vezes, limita o valor de saída a um máximo e um mínimo, como a função mostrada no gráfico abaixo. Escreva uma função em Python que recebe um número x por argumento e retorna o y equivalente, onde y é dado pela seguinte função por partes:



Resposta:

```
def funcao_de_ativacao( x ):

    if x > 1: # y = 1, saturação positiva
        return 1
    elif x < -1: # y = -1, saturação negativa
        return -1
    else: # região linear
        return x
```

Q3. Considere um jogo no qual um jogador tem a tarefa de encontrar o maior número de bolas vermelhas dentro de um cesto com bolas de várias cores no menor tempo possível. O jogador pode finalizar a tarefa a qualquer momento, mesmo que nem todas as bolas tenham sido encontradas. A pontuação base do jogador é dada pela seguinte regra:

- Se o jogador encontrar todas as bolas vermelhas disponíveis, então ele ganha 100 pontos.
- Se o jogador não encontrar todas as bolas vermelhas disponíveis, mas achar mais do que 70% desse total, então ele ganha 50 pontos.
- Se o jogador encontrar entre 30% (inclusive) e 70% (inclusive) do total de bolas vermelhas disponíveis, então ele ganha 25 pontos.
- Se o jogador encontrar menos de 30% das bolas vermelhas disponíveis, então ele não pontua.

A pontuação final do jogador é definida segundo o tempo que ele levou para concluir a tarefa, e segue a seguinte regra:

- Se o jogador concluiu a tarefa até antes do tempo limite estipulado (inclusive), então a pontuação final é igual à pontuação base.
- Se o jogador concluiu a tarefa até 10 minutos depois do tempo limite estipulado, então a pontuação final é 80% da pontuação base obtida.
- Se o jogador concluiu a tarefa entre 10 (inclusive) e 30 (inclusive) minutos depois do tempo limite estipulado, então a pontuação final é 40% da pontuação base obtida.
- Se o jogador concluiu a tarefa mais de 30 minutos depois do tempo limite estipulado, então o jogador não pontua.

Escreva uma função em Python que calcule a pontuação final de um jogador neste jogo. A função recebe 4 argumentos inteiros, nesta ordem: a quantidade de bolas vermelhas encontradas pelo jogador, a quantidade total de bolas vermelhas totais estavam no cesto, o tempo limite da tarefa (em minutos), e o tempo em que o jogador concluiu a tarefa (em minutos). A função deve retornar um número que representa a pontuação final do jogador.

Resposta:

```
def calcular_pontuacao_final(encontradas, total, tempo_limite, tempo_conclusao):

    porcentagem = encontradas/total
    pontos = 0

    # verificando pontos
    if encontradas == total:
        pontos = 100
    elif porcentagem > 0.7:
        pontos = 50
    elif porcentagem >= 0.3 and porcentagem <= 0.7:
        pontos = 25
    else: # porcentagem < 30 %
        pontos = 0

    fator_de_penalizacao = 1
    atraso = tempo_conclusao - tempo_limite

    # penalizando o jogador pelo tempo
    if atraso <= 0:
        fator_de_penalizacao = 1
    elif atraso < 10:
        fator_de_penalizacao = 0.8
    elif atraso >= 10 and atraso <= 30:
        fator_de_penalizacao = 0.4
    else:
        fator_de_penalizacao = 0

    return pontos * fator_de_penalizacao
```

- Q4.** (Inspirada na OBI 2010 - Nível 1, Fase 1) A empresa local de abastecimento de água, a Saneamento Básico da Cidade (SBC), está promovendo uma campanha de conservação de água, distribuindo cartilhas e promovendo ações demonstrando a importância da água para a vida e para o meio ambiente. Para incentivar mais ainda a economia de água, a SBC alterou os preços de seu fornecimento de forma que, proporcionalmente, aqueles clientes que consumirem menos água paguem menos pelo metro cúbico. Todo cliente paga mensalmente uma assinatura de R\$ 10, que inclui uma franquia de 15 m³ de água. Isto é, para qualquer consumo entre 0 e 15 m³, o consumidor paga a mesma quantia de R\$ 10 reais (note que o valor da assinatura deve ser pago mesmo que o consumidor não tenha consumido água). Acima de 15 m³ cada

metro cúbico subsequente tem um valor diferente, dependendo da faixa de consumo. A tabela abaixo especifica o preço por metro cúbico para cada faixa de consumo:

Faixa de consumo (m ³)	Preço (por m ³)
Até 15	Incluído na franquia
Acima de 15 a 40	R\$ 1.2
Acima de 40	R\$ 3.0

Por exemplo, se o consumo foi de 55 m³, o valor da conta é dado por:

- 10 reais da assinatura básica;
- 30 reais pelo consumo no intervalo acima de 15 até 40 m³;
- 45 reais pelo consumo no intervalo acima de 40 até 55 m³;

Isto é: $10 + 25 \times 1.2 + 15 \times 3.0 = \text{R\$}85.0$.

Escreva uma função em Python para calcular o valor total a ser pago por uma família. Essa função recebe por argumento três números inteiros que correspondem aos valores do consumo de água das três residências da família, em m³, e retorna o valor total da conta de água (soma dos valores a serem pagos em cada residência).

Antes de fazer a conta, no entanto, a função deve verificar se os argumentos de entrada são válidos: os argumentos devem ser do tipo inteiro ou float. Se algum dos valores passados por argumento não for nem inteiro nem float, retorne -1. Se todos os valores forem inteiros ou float, verifique se a faixa está correta: os valores devem ser maiores que 0 e menores que 5000. Se algum dos valores estiver fora dessa faixa, retorne -2. Se todos os valores forem inteiros ou floats e estiverem dentro da faixa, verifique o consumo e retorne o total. Utilize operadores lógicos booleanos (**not**, **and** ou **or**) em pelo menos uma das condicionais.

Outro exemplo: se a entrada for (15.5, 41, 60), a saída deve ser 153.6, calculado da seguinte forma:

$$(10 + 0.5 \times 1.2) + (10 + 25 \times 1.2 + 1 \times 3) + (10 + 25 \times 1.2 + 20 \times 3.0)$$

onde $(10 + 0.5 \times 1.2)$ é referente ao valor do consumo da primeira residência (15.5), $(10 + 25 \times 1.2 + 1 \times 3)$ é referente ao valor do consumo da segunda residência (41) e $(10 + 25 \times 1.2 + 20 \times 3.0)$ é referente ao valor do consumo da terceira residência (60).

Resposta:

0 =< 15m3 , valor = 10

> 15m3 e <= 40 , 1.2 por m3 consumido a mais

>40m3, 3 por m3 consumido a mais

```
def calcular_conta_individual( consumo ):  
    if (not type(consumo) == int) or (not type(consumo) == float):  
        return -1  
  
    if not (0 < consumo_1 < 5000):
```

```

        return -2

    if consumo > 40:
        excedente = consumo - 40
        return 10 + 25*1.2 + excedente*3
    elif 15 < consumo <= 40:
        excedente = consumo - 15
        return 10 + excedente * 1.2
    else: # taxa padrão da empresa
        return 10

def calcular_conta_das_familias( consumo_1 , consumo_2 , consumo_3 ):

    conta_residencia_1 = calcular_conta_individual(consumo_1)

    if conta_residencia_1 == -1:
        print("Argumento consumo 1 não é inteiro ou float")
        return -1

    if conta_residencia_1 == -2:
        print(f"Consumo 1 ({consumo_1}) fora da faixa permitida.")
        return -2

    conta_residencia_2 = calcular_conta_individual(consumo_2)

    if conta_residencia_2 == -1:
        print("Argumento consumo 2 não é inteiro ou float")
        return -1

    if conta_residencia_2 == -2:
        print(f"Consumo 2 ({consumo_2}) fora da faixa permitida.")
        return -2

    conta_residencia_3 = calcular_conta_individual(consumo_3)

    if conta_residencia_3 == -1:
        print("Argumento consumo 3 não é inteiro ou float")
        return -1

    if conta_residencia_3 == -2:
        print(f"Consumo 3 ({consumo_3}) fora da faixa permitida.")
        return -2

    consumo_total = consumo_1 + consumo_2 + consumo_3

```

```
valor_total = conta_residencia_1 + conta_residencia_2 + conta_residencia_3
print(f"O valor consumido pela familia é de {consumo_total} m3.")
print(f"Valor a ser pago é de {valor_total} reais")
return valor_total
```